# Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl

SANTO TOMÁS EDUCACIÓN CONTINUA

# Introducción a la Automatización en CDG

- **Eliminación de fricción operativa:** Erradicar tareas manuales rutinarias que consumen tiempo y no aportan valor analítico real al negocio.
- **Puntualidad ejecutiva:** Garantizar que los reportes clave lleguen a la bandeja de entrada de los gerentes de manera impecable a primera hora del día.
- **Integridad de datos:** Reducir drásticamente a cero el margen de error humano en la extracción, procesamiento y distribución de información contable.

# ¿Qué es la Automatización de Scripts?

- **Autonomía operativa:** Proceso mediante el cual configuramos el sistema operativo para ejecutar un script de Python en intervalos de tiempo programados, operando de manera 100 % transparente, desatendida y sin intervención humana.
- **Arquitectura de fondo:** Combina un lenguaje de programación versátil (Python) con un disparador a nivel de sistema operativo (como Task Scheduler en Windows o Cron en Linux).

# La Librería de Envío de Correos: Smtplib

- **Protocolo estándar de red:** Módulo nativo de la librería estándar de Python diseñado específicamente para interactuar con servidores de correo electrónico utilizando el protocolo SMTP (*Simple Mail Transfer Protocol*).
- Permite establecer sockets de red, gestionar autenticaciones seguras y transmitir los flujos de datos del mensaje.

# La Librería de Contenido: Email

- **Construcción de mensajes MIME:** Conjunto de librerías nativas destinadas a estructurar correos electrónicos avanzados, soportando texto enriquecido, plantillas HTML y la inyección segura de archivos adjuntos (PDFs o planillas de Excel corporativas).

# Configuración de la Conexión SMTP

- Establecer la comunicación inicial de red con los servidores corporativos o proveedores de correo (ej. Gmail) mediante el puerto seguro TLS/SSL.

```
import smtplib

# Inicializar conexi n con servidor SMTP (ej. Gmail en puerto 587)
server = smtplib.SMTP('smtp.gmail.com', 587)
```

# Inicio del Protocolo Seguro (StartTLS)

- Elevar la conexión estándar de red a un canal cifrado para garantizar que las credenciales corporativas viajen protegidas contra interceptaciones.

```
# Activar cifrado de capa de transporte (TLS)
server.starttls()
```

# Autenticación SMTP

- Validar la identidad del remitente utilizando credenciales institucionales o contraseñas de aplicación dedicadas para evitar bloqueos de seguridad.

```
usuario = "reportes@empresa.com"
contrasenia = "clave_de_aplicacion_segura"

server.login(usuario, contrasenia)
```

# Creación de un Mensaje Multipart MIME

- Estructurar el contenedor principal del correo, definiendo metadatos esenciales como el remitente, destinatario y asunto del reporte contable.

```
from email.mime.multipart import MIMEMultipart

msg = MIMEMultipart()
msg['From'] = "reportes@empresa.com"
msg['To'] = "gerencia@empresa.com"
msg['Subject'] = "Reporte Financiero Diario - Automatizado"
```

# Cuerpo del Mensaje en Formato HTML

- Incorporar contenido enriquecido utilizando etiquetas HTML para dotar al reporte de un aspecto visual corporativo y profesional.

```
from email.mime.text import MIMEText

html_content = "<h2>Informe Diario</h2><p>Adjunto encontrar el balance.</p>"
msg.attach(MIMEText(html_content, 'html'))
```

# Incrustar Archivos Adjuntos

- Leer archivos locales en modo binario, codificarlos en base64 e inyectarlos de manera segura como adjuntos MIME.

```
from email.mime.base import MIMEBase
from email import encoders

with open("reporte_financiero.pdf", "rb") as adjunto:
    part = MIMEBase("application", "octet-stream")
    part.set_payload(adjunto.read())
    encoders.encode_base64(part)
    part.add_header("Content-Disposition", "attachment; filename=reporte.pdf
    ")
    msg.attach(part)
```

# Flujo Completo del Envío de Correos

Secuencia lógica obligatoria en la ejecución del script:

1. **Conectar** con el servidor SMTP mediante socket.
2. **Encriptar** el canal de comunicación (`starttls`).
3. **Autenticar** las credenciales institucionales.
4. **Construir** el mensaje y adjuntar archivos analíticos.
5. **Enviar** el correo electrónico corporativo.
6. **Cerrar** explícitamente la conexión de red.

# El Programador de Tareas de Windows (Task Scheduler)

- Herramienta nativa del sistema operativo Windows que permite automatizar la ejecución de scripts o programas en función de intervalos de tiempo, horarios específicos o eventos del sistema.

# Creación de un Script Ejecutable (.bat)

- Para ejecutar scripts de Python en segundo plano desde el sistema operativo de forma limpia, se encapsulan las rutas del intérprete y del archivo de código en un archivo por lotes (`.bat`).

# Estructura de un Archivo .bat

Contenido del script por lotes que invoca el entorno y el código analítico:

```
@echo off
"C:\Program Files\Python310\python.exe" "C:\scripts\send_kpi.py"
pause
```

# Configuración en Task Scheduler: El Activador

- Definición del disparador temporal: Configurar la tarea para que se gatille de forma diaria, semanal o en un horario estricto (ej. todos los días laborables a las 08:00 AM).

# Configuración en Task Scheduler: La Acción

- Establecer la instrucción del sistema: Seleccionar la opción *'Iniciar un programa'* y apuntar directamente a la ruta absoluta del archivo `.bat` previamente creado.

# Manejo de Credenciales Seguras

- **Buena práctica de ciberseguridad:** Nunca almacenar contraseñas en texto plano dentro del código fuente de Python. Se deben delegar al entorno del sistema operativo.

# Uso de la Librería Os para Variables de Entorno

Extracción segura de claves secretas directamente desde las variables del sistema operativo:

```
import os

# Obtener credenciales seguras configuradas en el servidor
usuario = os.getenv('SMTP_USER')
contrasenia = os.getenv('SMTP_PASSWORD')
```

# Monitoreo de Ejecuciones Exitosas

- **Auditoría local:** Generar registros de eventos en archivos de texto locales (`.log`) para certificar si el reporte diario se despachó con éxito o si se interrumpió por fallos.

# Escritura de Registros en Archivos de Log

Ejemplo de implementación para registrar la traza temporal de ejecuciones:

```
import datetime

timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
with open('registro_auditoria.log', 'a') as f:
    f.write(f"[{timestamp}] - Reporte enviado exitosamente.\n")
```

# Manejo de Errores Críticos de Red

- Implementar bloques de control de excepciones y reintentos automatizados ante intermitencias temporales en el servidor SMTP corporativo.

```
try:
    server.sendmail(remitente, destinatario, msg.as_string())
except Exception as e:
    print(f"Error de red cr tico: {e}")
```

# Envío Condicional de Alertas por Desviaciones

- Evaluar KPIs financieros mediante Pandas y gatillar el correo electrónico únicamente ante desvíos críticos presupuestarios.

```
if sobrepresupuesto_porcentaje > 10.0:
    enviar_alerta_gerencial(msg)
```

# Optimización de Archivos Adjuntos Pesados

- Comprimir reportes extensos en formato ZIP antes del envío para cumplir con las políticas de tamaño máximo de las bandejas de correo corporativas.

# Resumen Ejecutivo de Planificación del Módulo

- Consolidación de aprendizajes: Python actúa como el motor central que conecta la extracción de datos, el análisis financiero, la generación de reportes y su distribución automatizada.

# Caso Práctico de Cierre del Módulo

- **Objetivo del entregable final:** Desarrollar un script integral en Python que procese datos con Pandas, detecte centros de costo fuera de norma, compile un PDF y ejecute el envío automatizado por SMTP.

# Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl

SANTO TOMÁS EDUCACIÓN CONTINUA